

PROGRAMMING & METHODS · THE AI API

Teaching my website to write

How every new post on The Iceberg gets a summary written by an AI

Dylan · Berghs · 2026

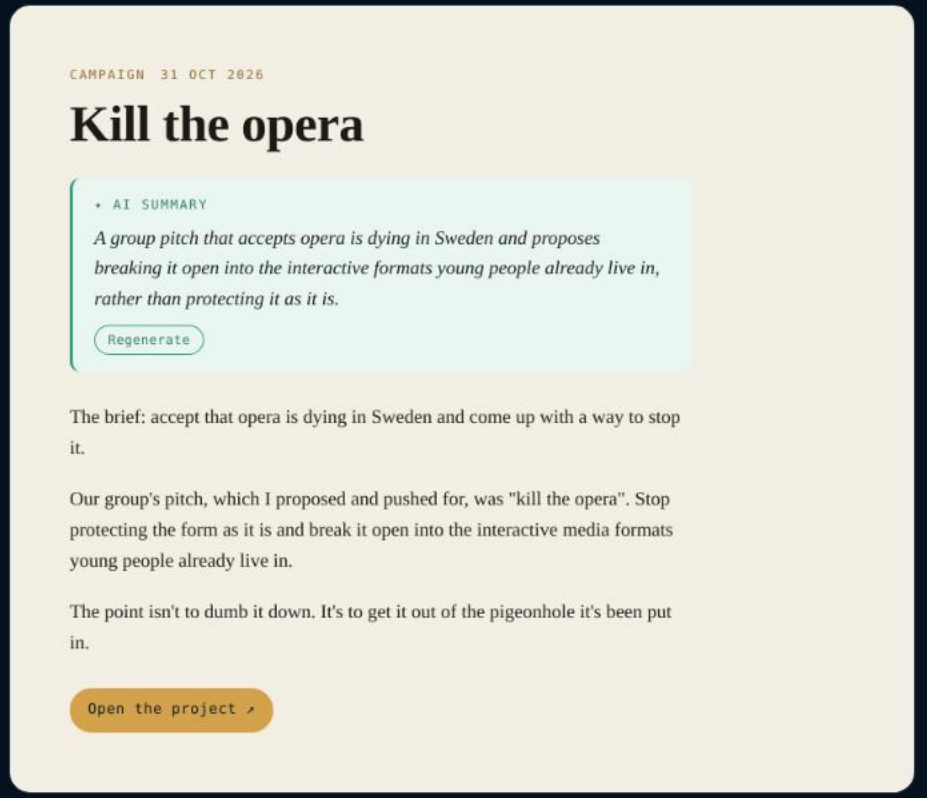
THE IDEA

What is an AI API?

An **API** is a way for one program to ask another program to do something for it.

An **AI API** sends some text to an AI model and gets new text back.

On my site: I publish a post, and a short summary appears on it, written by an AI.



TODAY

What you’ll learn

01

The journey of one request

02

What goes into the request

03

What comes back

04

How to keep the key secret

HOW IT WORKS

One request, from my page to the AI and back

Browser

I press Publish

My server

Holds the API key. Saves the post in Neon.

OpenRouter

Checks the key, forwards it

AI model

Writes the summary

The summary travels back the same way: model → OpenRouter → my server, which saves it in the database and sends it to the page.

FIVE WORDS TO KNOW

The vocabulary

Word	What it means here
API key	A password that proves the request is mine, and the account that gets billed
Model	Which AI does the writing. I use the latest Claude Sonnet, through OpenRouter
Prompt	The messages I send: a **system** message with my rules, a **user** message with the post
Token	A small chunk of text, about three quarters of a word. You pay per token
Temperature	How adventurous the wording is: low = similar every time, high = more creative

FROM MY PROJECT · THE ILLEGAL ARCHIVE · server/ai.js

What I send

server/ai.js

```
20  const response = await fetch(OPENROUTER_URL, {
21    method: 'POST',
22    headers: {
23      Authorization: `Bearer ${process.env.OPENROUTER_API_KEY}`,
24      'Content-Type': 'application/json',
25      'X-OpenRouter-Title': 'The Logbook',
26    },
27    body: JSON.stringify({
28      model: MODEL,
29      max_tokens: 120,          // a hard cap on reply length (and cost)
30      temperature: 0.4,         // lower = more predictable, less "creative"
31      messages: [
32        {
33          role: 'system',
34          content:
35            'You summarise blog posts for a creative portfolio journal. ' +
36            'Write ONE or TWO plain sentences, max 40 words, in the same language as the post. ' +
37            'Third person is not needed; write it as a neutral teaser. ' +
38            'No headings, no quotes, no emojis, no preamble like "This post".',
39        },
40        { role: 'user', content: `Title: ${title}\n\n${body}` },
41      ],
42    }),
43    signal: AbortSignal.timeout(20000),   // give up after 20 seconds
44  });
```

`Line 23`

The **key** goes in a header, read from my secret **.env** file

`Line 28`

Which **model** should write it

`Lines 29–30`

Limits: at most 120 tokens, and a low temperature so it stays factual

`Lines 33–40`

Two messages: my rules (one or two sentences, max 40 words), then the post

FROM MY PROJECT · THE ILLEGAL ARCHIVE · server/ai.js

What comes back

server/ai.js

```
46   if (!response.ok) {
47     const detail = await response.text();
48     throw new Error(`OpenRouter error ${response.status}: ${detail.slice(0, 200)}`);
49   }
50
51   const data = await response.json();
52   const text = data.choices?.[0]?.message?.content?.trim();
53   return text || null;
54 }
```

`choices → [0] → message → content`

The reply is JSON. The summary text sits a few levels deep, so I dig it out. The **?.** stops it crashing if a part is missing.

`response.ok = false`

Wrong key, no credit or OpenRouter down: I throw an error with their message, so I can see what went wrong.

FROM MY PROJECT · THE ILLEGAL ARCHIVE · server/routes/posts.js

When it runs: right after a post is saved

server/routes/posts.js

```
112     // Ask the AI for a summary. If it fails, the post is still saved:
113     // the summary is a bonus, not something that should block publishing.
114     try {
115       post.summary = await summarise(title, body);
116       if (post.summary) {
117         await query('UPDATE posts SET summary = $1 WHERE id = $2', [post.summary, post.id]);
118       }
119     } catch (err) {
120       console.error('AI summary failed:', err.message);
121     }
122
123     res.status(201).json(withVideo(post));
```

`Line 115 · 117`

Ask the AI, then store the answer next to the post with one SQL **UPDATE**

`try … catch`

If the AI fails, the post is still published. The summary is a bonus, never a blocker.

FROM MY PROJECT · THE ILLEGAL ARCHIVE · public/js/blog.js

Regenerate, from the page

public/js/blog.js

```
204   // Signed-in crew get a button to (re)generate it
205   if (currentUser) {
206     const button = el('button', { class: 'btn ghost small', type: 'button' },
207       post.summary ? 'Regenerate' : 'Generate summary');
208     button.addEventListener('click', async () => {
209       button.disabled = true;
210       button.textContent = 'Thinking…';
211       try {
212         const { summary } = await api.summarise(post.id);
213         text.textContent = summary;
214         button.textContent = 'Regenerate';
215       } catch (err) {
216         text.textContent = err.message;
217         button.textContent = 'Try again';
218       } finally {
219         button.disabled = false;
220       }
221     });
222     box.append(button);
```

Signed-in, I get a **Regenerate** button on every post.

Line 212: the browser asks **my server**, never OpenRouter.

While it waits the button says Thinking…, and errors show on the page instead of breaking it.

THE MOST IMPORTANT PART

FROM MY PROJECT · THE ILLEGAL ARCHIVE · .env.example + .gitignore

The key never reaches the browser

.env.example

```
10  # openrouter.ai → Keys
11  OPENROUTER_API_KEY=sk-or-your-key-here
12  # Optional: which AI model writes the summaries
13  OPENROUTER_MODEL=~anthropic/claude-sonnet-latest
```

.gitignore

```
1  node_modules/
2  .env
3  .DS_Store
```

The real key lives in **.env**, which **.gitignore** keeps off GitHub. **.env.example** shows the shape with a fake key.

If the key were in browser code, anyone could open DevTools, copy it and spend my credit.

How I built it

Docs I used

OpenRouter’s quick-start page: the address, the headers, and where the reply text sits.

AI as my pair programmer

I built it with Claude, one stage at a time, testing each stage in the browser before committing it.

Choosing a model

OpenRouter gives one API for many models. Mine is a setting in .env, so I can swap it without touching the code.

Problems I hit

OpenRouter wasn’t reachable from the coding environment, so I first tested the whole flow against a fake server that answers in the same format.

RECAP

01 **The journey:** browser → my server → OpenRouter → AI model, and back

02 **Send** the key, a model, limits and two messages. **Read** choices[0].message.content

03 **Keep the key** on the server, in .env, out of Git

Questions?